Text preprocessing

1. RE
2. Data exploration (Heap's law)
3. MED

① RE : disjunction / Negation / Or / Others

word tokenization (spaCy)

② Heap's law

$V = \{v_1, v_2, \cdots, v_{|V|}\}$ vocab

$|V| = k \cdot T^{\beta}$ $\begin{cases} k \in [10, 100] \\ \beta \in [0.4, 0.6] \\ T : \text{\# of tokens} \end{cases}$

③ MED :

$X = [x_1, x_2, \cdots, x_n]$

$Y = [y_1, y_2, \cdots, y_m]$

- insert (Ins) cost 1
- delete (Del) cost 1
- substitution (Sub) cost 2

$O_k = [O_1, O_2, \cdots, O_k]$ $O_i \in \{Ins, Del, Sub\}$

$X \xrightarrow{O_k} Y$: $\min[\text{cost}(O_k)]$

Dynamic programming

$X_i = [x_1, x_2, \cdots, x_i]$, $i = 0, 1, 2, \cdots, n$

$Y_j = [y_1, y_2, \cdots, y_j]$, $j = 0, 1, 2, \cdots, m$

Sub problem : $X_i \to Y_j$: $D[i, j]$: MED of $(X_i \to Y_j)$

- $D[i,0] = i$ (delete all i chars from X_i)
- $D[0,j] = j$ (insert all j chars into Y_j)

DP: <u>optimal substructure</u>

Let $O_k = [O_1, O_2, \cdots, O_k]$ be minimal operation seq.

$X_i \to Y_j$

$$X_i = X_i^0 \xrightarrow{O_1} X_i^1 \xrightarrow{O_2} X_i^2 \cdots \xrightarrow{O_{k-1}} X_i^{k-1} \xrightarrow{O_k} X_i^k = Y_j$$

remove last: O_k

$O_{k-1} = [O_1, O_2, \cdots, O_{k-1}]$: optimal $X_i \to X_i^{k-1}$

proof: say O_{k-1} is not optimal

Optimal O'_{k-1} : $cost(O'_{k-1}) < cost(O_{k-1})$

Then $O'_k = [O'_{k-1}, O_k]$

$cost(O'_k) < cost(O_k)$: contradiction

three possible options for O_k:

Case 1: delete x_i from X_i (X_i^{k-1})

$D[i,j] = D[i-1,j] + Del(x_i)$

Case 2: insert y_j into Y_{j-1} (X_i^{k-1})

$D[i,j] = D[i,j-1] + Ins(y_j)$

Case 3: substitute x_i by y_j

$D[i,j] = D[i-1,j-1] + Sub(x_i, y_j)$

$D[i, j] = \min \{ \text{case } 1, 2, 3 \}$: $O(n \cdot m)$

Remark: DP is important in classic NLP.

1. MED
2. HMM (Hidden Markov Models)
3. CKY parsing (context free grammar parsing)
4. CRF (Viterbi) : NER.